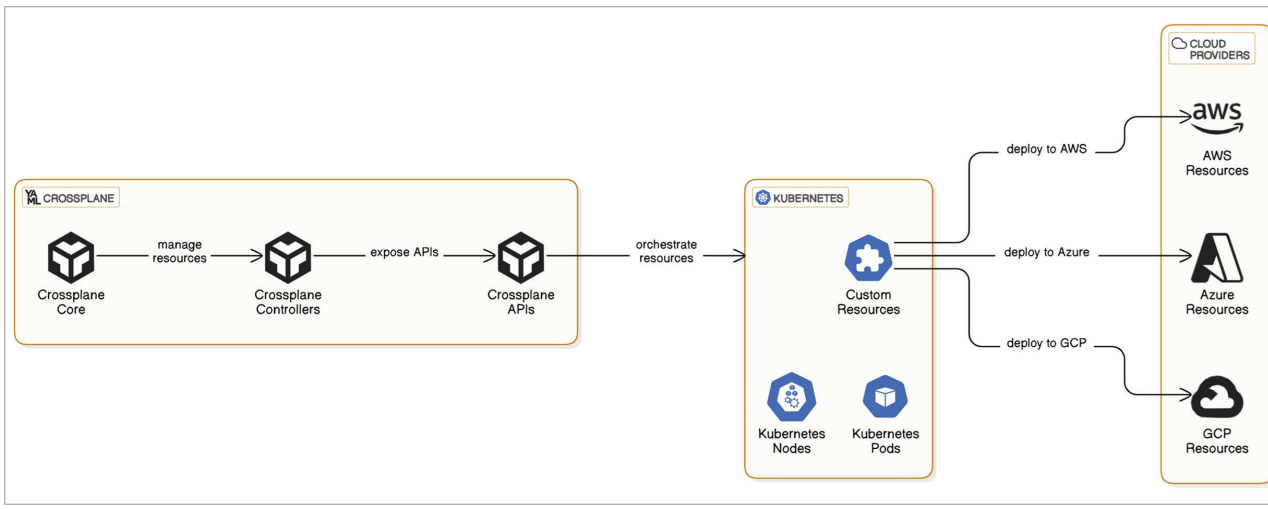

Figure 1: Reference architecture for multi-cloud deployment with Crossplane orchestration

manual intervention or external factors. This guarantees a high level of availability and minimises the operational overhead that is associated with manual interventions.

Reconciling controllers

Reconciling controllers are the foundation of Crossplane's functionality, as they are responsible for managing the lifecycle of resources. The configuration manifests specify the intended state, which these controllers continuously reconcile with the actual state of the infrastructure. In doing so, they guarantee that the infrastructure remains in a consistent and predictable state. Crossplane capitalises on the control loop mechanism of Kubernetes, which enables controllers to monitor modifications to resource specifications and implement the actions needed to reconcile the current state with the desired state. This automation mitigates the risk of configuration drift and improves the system's overall stability.

Extensibility

One of the most noteworthy attributes of Crossplane is its extensibility. The framework can be expanded by incorporating the concepts of Configuration Packages and Composite Resource Definitions (XRDs). Custom resources that represent higher-level abstractions of infrastructure components, such as databases, networks, or complete application stacks, can be defined by users using XRDs. On the other hand, Configuration Packages facilitate the grouping of numerous resource definitions and their respective controllers into reusable units. This modularity enables organisations to customise Crossplane to meet their unique requirements and seamlessly integrate it with their current procedures and tools.

Extending Kubernetes for infrastructure as code

Crossplane augments Kubernetes' capabilities to provision and administer infrastructure as code (IaC) across a variety of cloud platforms, such as Google Cloud, Azure, and AWS. It incorporates infrastructure management into the Kubernetes ecosystem by defining infrastructure resources as Kubernetes Custom Resource Definitions (CRDs). This method allows developers to utilise a unified set of tools and practices to administer both application and infrastructure resources. Users can provision and administer infrastructure in a cloud-agnostic manner using Crossplane's provider ecosystem, which supports a diverse array of cloud services.

Building cloud-agnostic and cloud-native solutions

Crossplane's capacity to facilitate the development of cloud-native and cloud-agnostic solutions is one of its primary advantages. Crossplane enables developers to construct applications that are not inextricably linked to any specific cloud provider by abstracting cloud-specific details and offering a standardised interface for infrastructure management. This decoupling improves portability and flexibility, allowing organisations to implement multi-cloud strategies and circumvent vendor lock-in. Furthermore, Crossplane's integration with Kubernetes guarantees that applications are developed from the ground up in accordance with cloud-native principles, including observability, resilience, and scalability.

Prerequisites for Crossplane

Ensure that the following are present in order to develop infrastructure as code using the Crossplane platform solution:

- Kubernetes cluster (private or public cloud)
- kubectl has been deployed and configured